

Лекция. ОПТИМИЗАЦИЯ НУЛЕВОГО ПОРЯДКА

1. Введение

Оптимизация — это одна из фундаментальных дисциплин прикладной математики, лежащая в основе всех современных вычислительных технологий. Под оптимизацией понимается процесс нахождения наилучшего решения из множества возможных, при этом критерий «наилучшего» определяется специальной функцией — целевой функцией или функцией потерь. С практической точки зрения, это попытка ответить на вопрос: «Как достичь максимума эффективности или минимума затрат при заданных ограничениях?»

Исторически задачи оптимизации возникали задолго до появления компьютеров. Уже в античные времена Архимед и Герон формулировали задачи на нахождение кратчайшего пути света или минимального объёма при заданной поверхности. В XVIII–XIX веках Лагранж и Эйлер заложили основы вариационного исчисления, позволившего находить экстремумы функционалов — то есть оптимальные функции, а не просто числа. В XX веке развитие вычислительной техники и появление понятий градиента, гессиана, итерационного спуска и численных методов сделало оптимизацию основным инструментом в инженерии и науке о данных.

В машинном обучении понятие оптимизации имеет центральное значение. Почти каждая обучаемая модель — от простой линейной регрессии до глубоких нейронных сетей — формулируется как задача минимизации некоторой функции потерь (cost function, loss function). Например, в линейной регрессии параметры $w = (w_0, w_1)$ — наклон и пересечение прямой — выбираются так, чтобы минимизировать квадратичную ошибку между прогнозируемыми и реальными значениями:

$$L(w_0, w_1) = (1/n) \sum (y_i - (w_1 x_i + w_0))^2$$

Точка (w_0^*, w_1^*) , в которой функция потерь принимает минимальное значение, является оптимальной в смысле задачи аппроксимации данных. Этот пример — конкретная реализация общей идеи: поиск параметров, делающих модель «лучшей» в соответствии с выбранным критерием качества. Похожим образом устроены и задачи классификации, где функция потерь измеряет ошибку разделения классов, или задачи кластеризации, где минимизируется внутрикластерная дисперсия.

Рассмотрим произвольную функцию многих переменных:

$$f(w): \mathbb{R}^N \rightarrow \mathbb{R}$$

Задача оптимизации формулируется как:

$$w^* = \arg \min f(w)$$

или, при максимизации:

$$w^* = \arg \max f(w)$$

В практических задачах аналитическое нахождение минимума почти никогда невозможно. Поэтому основное внимание уделяется численным алгоритмам, которые итерационно приближаются к точке минимума, изменяя параметры по определённым правилам.

Глобальный минимум — это точка, где значение функции меньше, чем в любой другой точке области определения. Локальный минимум — точка, где значение функции меньше, чем у соседних точек. Многие функции имеют несколько локальных минимумов и максимумов; в машинном обучении часто невозможно отличить глобальный минимум от локального из-за сложной геометрии поверхности потерь.

Методы нулевого порядка — это алгоритмы оптимизации, не требующие вычисления производных функций. Они основаны лишь на сравнении значений функции в разных точках. Эти методы особенно полезны, когда функция не имеет аналитического выражения, содержит шум или разрывы, а производные вычислить невозможно или слишком дорого.

Связь с другими областями: в экономике — нахождение равновесных цен, минимизация затрат; в инженерии — оптимизация параметров систем и конструкций; в биологии — моделирование эволюции; в логистике — маршруты и запасы; в информатике — задачи поиска и планирования. Оптимизация — универсальный язык науки и техники.

ВЫВОД: изучая методы нулевого порядка, мы делаем первый шаг к пониманию всей иерархии методов оптимизации. Они просты, но позволяют увидеть ключевые идеи: поиск минимума, направление движения, значение шага и принцип последовательного улучшения решения. В дальнейшем мы рассмотрим методы первого и второго порядка, использующие производные и кривизну функции.

2. Условие нулевого порядка оптимальности

Прежде чем переходить к алгоритмам, необходимо формально определить, что означает «минимум» и «максимум» функции без обращения к производным. Это даёт условие нулевого порядка оптимальности — самая базовая формализация экстремумов, опирающаяся только на значения функции.

2.1 Формальные определения

Пусть задана функция многих переменных:

$$f(\mathbf{w}) : \mathbb{R}^N \rightarrow \mathbb{R}$$

Здесь $w = (w_1, w_2, \dots, w_N)$ — вектор параметров, а $f(w)$ — скалярная величина, которую мы хотим минимизировать (или максимизировать).

Определение глобального минимума:

$$f(w^*) \leq f(w), \quad \forall w \in \mathbb{R}^N$$

Определение глобального максимума:

$$f(w^*) \geq f(w), \quad \forall w \in \mathbb{R}^N$$

Определение локального минимума вводится через окрестность точки. Существует $\delta > 0$ такая, что для всех w , удовлетворяющих $\|w - w^*\| < \delta$, выполняется:

$$f(w^*) \leq f(w), \quad \forall w : \|w - w^*\| < \delta$$

Аналогично для локального максимума:

$$f(w^*) \geq f(w), \quad \forall w : \|w - w^*\| < \delta$$

Замечание: максимум функции f совпадает с минимумом функции $-f$. Поэтому многие рассуждения достаточно вести только для минимума.

2.2 Интуитивные примеры

Пример 1 (квадратичная функция). Рассмотрим параболу:

$$f(x) = (x - 2)^2 + 1$$

Минимум достигается при $x = 2$ и равен $f(2) = 1$. Если рассмотреть функцию $-f(x)$, график перевернётся, и точка $x = 2$ станет глобальным максимумом.

Пример 2 (синусоида):

$$f(x) = \sin(x)$$

Глобальные максимумы расположены в точках $x = \pi/2 + 2\pi k$, глобальные минимумы — в точках $x = 3\pi/2 + 2\pi k$, $k \in \mathbb{Z}$. Пример демонстрирует, что число экстремумов может быть бесконечным.

2.3 Понятие локальности

Во многих задачах функция имеет несколько впадин разной глубины. Локальный минимум — это точка, значение функции в которой меньше, чем у всех «ближайших» точек, но не обязательно меньше, чем в других, удалённых областях. Формально это и выражается через условие на окрестность с радиусом δ .

Интуитивно локальный минимум — «яма» на поверхности функции, тогда как глобальный минимум — «самая глубокая яма» на всём пространстве определения.

2.4 Почему «нулевой» порядок

Терминология «нулевой порядок» (zero-order) пришла из математического анализа: когда мы не используем ни первую, ни вторую производные, а опираемся только на значение функции. В отличие от этого методы первого порядка используют градиент $\nabla f(w)$, а второго — гессиан $\nabla^2 f(w)$.

Преимущество нулевого порядка — применимость к случаям, когда функция недифференцируема, содержит шум или известна только в виде чёрного ящика (результата эксперимента или симуляции).

2.5 Геометрическая интерпретация

Пусть $f(x, y)$ задана на плоскости. Её график — поверхность над координатной плоскостью. Глобальный минимум — самая низкая точка поверхности; локальный — точка, ниже всех в некоторой окрестности. Если при любом малом сдвиге из точки значение функции не становится меньше, то точка является минимумом нулевого порядка.

2.6 Практическая значимость

Условия нулевого порядка лежат в основе критериев, реально используемых в алгоритмах: мы сравниваем значения $f(w)$ в кандидатных точках и принимаем решение о переходе, если наблюдаем уменьшение. Классические примеры использования: глобальная оптимизация перебором/выборкой, локальные нисходящие шаги, критерии остановки по отсутствию уменьшения значения функции.

В дальнейшем эти определения становятся фундаментом для методов первого/второго порядка, где уже используются градиенты и кривизна, но логика проверки улучшения всё равно опирается на сравнение значений функции.

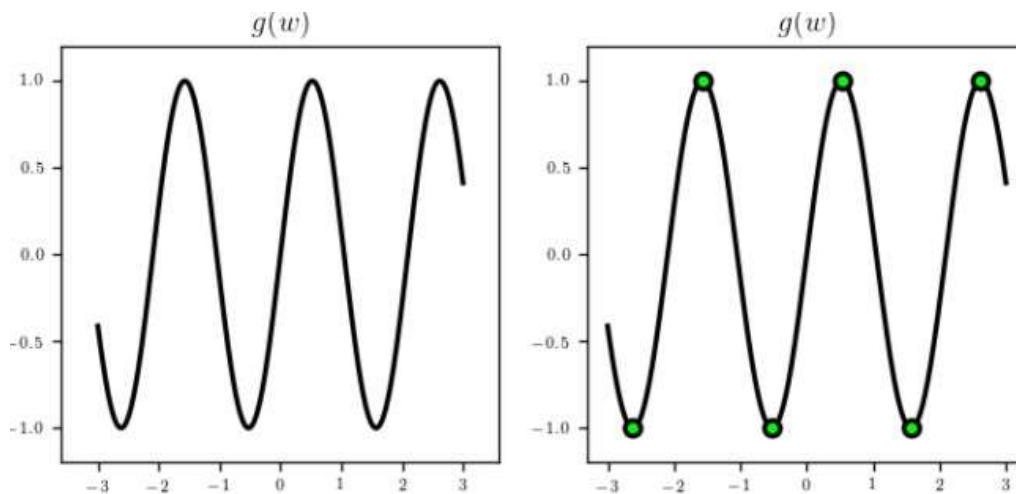


Рис. 2.2 — Глобальные и локальные экстремумы функции.

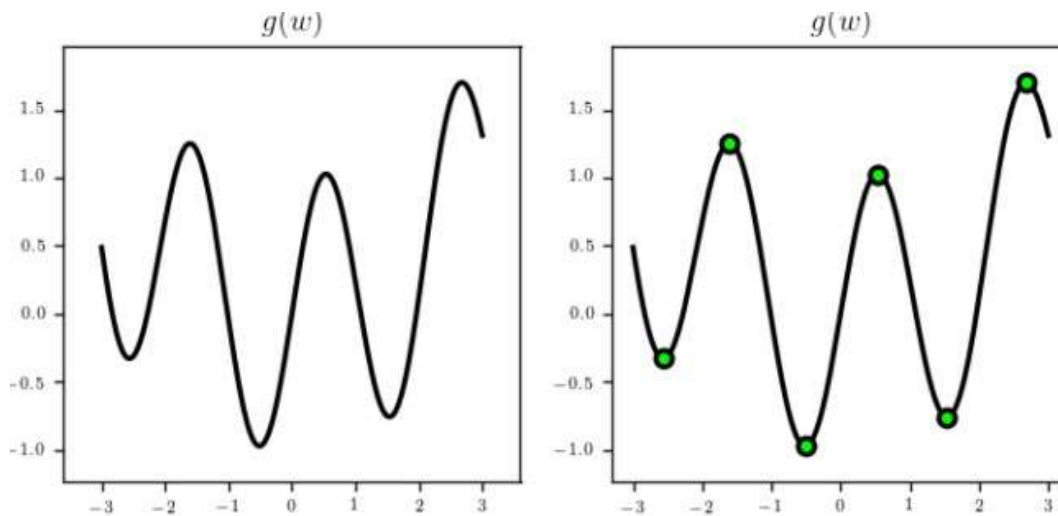


Рис. 2.3 — Пример функции с несколькими локальными минимумами (сумма синусоиды и параболы).

3. Методы глобальной оптимизации

Методы глобальной оптимизации представляют собой первый шаг к алгоритмическому поиску экстремума функции. В отличие от локальных методов, которые уточняют решение вблизи начальной точки, глобальные методы оценивают функцию во множестве точек пространства и выбирают ту, где значение минимально. Они не используют производные и полностью основаны на нулевом порядке — сравнении значений функции.

3.1 Основная идея

Основная идея метода проста: вычислить значение функции в ряде выбранных точек и определить, где функция принимает наименьшее значение. Такой подход формально выражается как:

$$w^* = \arg \min_{i=1, \dots, K} f(w_i)$$

где w_i — точки из выбранного множества выборки, а $f(w_i)$ — соответствующие значения функции. Если количество точек K достаточно велико, найденный минимум приближает истинный глобальный минимум.

3.2 Визуальное понимание

Для функции одной переменной, например $f(x) = (x - 1)^2 + 0.5$, можно просто построить график и увидеть, что минимум находится при $x = 1$. В двумерном случае, например $f(x, y) = x^2 + y^2$, глобальный минимум расположен в точке $(0, 0)$. Если вычислить значения функции на сетке точек и выбрать минимальное, результат совпадёт с визуально наблюдаемым минимумом.

Такой метод можно рассматривать как численную имитацию человеческого восприятия: «посмотреть на поверхность функции» и найти самую низкую точку. В численной практике это реализуется как *grid search* (поиск по сетке) или *random sampling* (случайное выборочное тестирование).

3.3 Выбор точек для оценки

Так как в непрерывном пространстве существует бесконечное число точек, необходимо ограничиться конечной выборкой. Существует два основных подхода к выбору точек:

1. Равномерная сетка (*uniform grid sampling*) — равномерное разбиение области на дискретные шаги.

2. Случайная выборка (*random sampling*) — точки выбираются случайным образом из допустимой области.

В последнем случае выборка может быть равномерной или подчиняться какому-либо распределению, например:

$$x_i \sim \mathcal{U}[a, b], \quad i = 1, 2, \dots, K$$

3.4 Пример: минимизация квадратичной функции

Рассмотрим функцию двух переменных:

$$f(x, y) = x^2 + y^2$$

Её истинный минимум находится в точке $(0, 0)$. Если взять 20 случайных точек в квадрате $[-1, 1] \times [-1, 1]$, вычислить значения функции и выбрать минимальное, то найденная точка будет близка к $(0, 0)$. Увеличивая количество точек, мы повышаем точность приближения.

Таким образом, глобальная оптимизация реализует нулевое условие оптимальности:

$$w^* \approx \arg \min_{\{w \in \{w_1, \dots, w_K\}\}} f(w)$$

3.5 Проклятие размерности

Главная трудность глобальных методов — так называемое «проклятие размерности». Количество точек, необходимых для покрытия пространства, растёт экспоненциально с числом переменных N .

$$K = m^N$$

Например, если мы берём по три значения на каждой оси ($m = 3$): при $N = 1$ достаточно 3 точек; при $N = 2$ — уже 9; при $N = 3$ — 27. При $N = 10$ требуется $3^{10} = 59\,049$ вычислений функции, а при $N = 20$ — миллиарды. Даже случайная выборка не решает проблему, поскольку плотность точек в многомерном пространстве резко падает.

Таким образом, глобальные методы практически неприменимы при большой размерности пространства параметров. Они полезны для задач с 1–3 переменными, где можно визуализировать поверхность функции и гарантировать нахождение минимума.

3.6 Применение и ограничения

Методы глобальной оптимизации применяются при:

- подборе гиперпараметров модели (grid search);
- анализе формы функции потерь;
- малой размерности задач, когда вычисление функции недорого.

Однако при увеличении размерности эффективность резко падает. Поэтому глобальные методы используются в качестве предварительного шага перед локальными методами, которые уточняют найденное решение.

3.7 Вывод

Методы глобальной оптимизации представляют собой простейший способ нахождения экстремума — через прямое сравнение значений функции во множестве точек. Они позволяют интуитивно понять идею минимума и служат основой для последующего изучения локальных методов. Однако из-за проклятия размерности их использование ограничено низкоразмерными задачами.

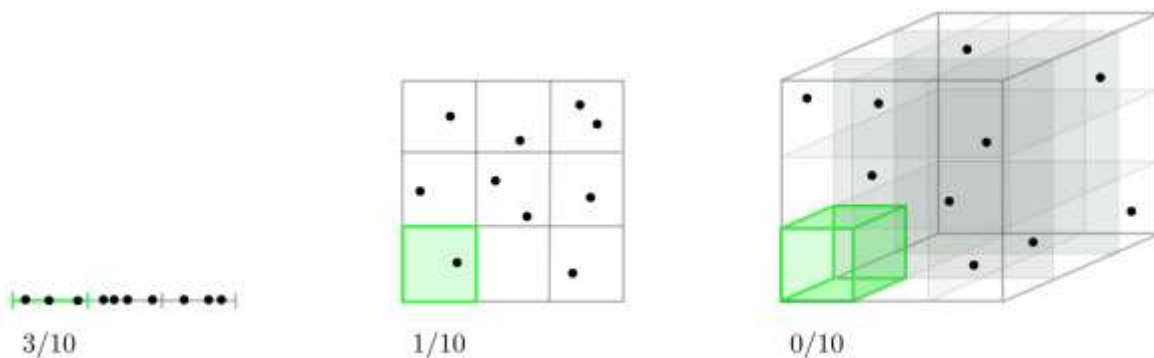


Рис. 3.1 — Иллюстрация проклятия размерности.

4. Методы локальной оптимизации

В отличие от глобальных методов, которые оценивают функцию в множестве точек, локальные методы оптимизации работают итерационно, начиная с одной точки и постепенно улучшая её. Главная идея — двигаться шаг за шагом в направлении уменьшения функции, пока не будет достигнут локальный минимум.

4.1 Общая схема локального метода

Пусть на k -й итерации имеем текущее приближение $w^k \in \mathbb{R}^N$. Чтобы получить следующее приближение w^{k+1} , выбираем направление d^k и длину шага $\alpha_k > 0$, после чего вычисляем:

$$w^{k+1} = w^k + \alpha_k d^k$$

Здесь d^k — направление спуска, а α_k — длина шага. Эти два параметра определяют, куда и насколько далеко перемещается точка на очередной итерации. В простейшем случае направление d^k выбирается так, чтобы значение функции уменьшалось: $f(w^k + \alpha_k d^k) < f(w^k)$.

4.2 Условие уменьшения функции

Чтобы движение было действительно нисходящим, необходимо выполнение условия уменьшения:

$$f(w^k + \alpha_k d^k) < f(w^k)$$

Если это условие не выполняется, шаг α_k уменьшают или выбирают другое направление. Таким образом формируется последовательность $\{w^k\}$, которая приближается к локальному минимуму функции.

4.3 Выбор направления спуска

Направление спуска — это любой вектор d^k , для которого при малом $\varepsilon > 0$ выполняется:

$$f(w^k + \varepsilon d^k) < f(w^k)$$

В методах нулевого порядка направление выбирается на основе сравнения значений функции — без вычисления градиента или производных. В методах первого порядка направление определяется по градиенту $\nabla f(w^k)$.

4.4 Роль длины шага

Длина шага α_k играет важную роль. Если шаг слишком велик, алгоритм может «перепрыгнуть» через минимум или начать колебаться. Если шаг слишком мал, сходимость будет крайне медленной.

Длина шага определяет расстояние между итерациями:

$$\|w^{k+1} - w^k\| = \alpha_k \|d^k\|$$

Чтобы обеспечить устойчивость, часто используют постоянный малый шаг ($\alpha_k = \text{const}$) либо убывающую последовательность шагов (например, $\alpha_k = 1/(k+1)$). В машинном обучении параметр α_k называют скоростью обучения (learning rate).

4.5 Пример нисходящей последовательности

Рассмотрим одномерную функцию $f(x) = (x - 2)^2$. Пусть начальная точка $x^0 = 5$, шаг $\alpha = 0.5$, направление $d = -1$. Тогда итерационный процесс запишется как:

$$x^{k+1} = x^k - 0.5$$

Последовательность $\{5, 4.5, 4.0, 3.5, \dots\}$ сходится к минимуму $x^* = 2$. При каждом шаге значение функции $f(x^k)$ уменьшается. Это пример нисходящей последовательности.

4.6 Геометрическая интерпретация

Локальные методы можно визуализировать как движение точки по поверхности функции вниз вдоль направления спуска. В двумерном случае траектория обычно изображается на карте линий уровня (contour plot), где каждая линия — это множество точек с одинаковым значением функции. Алгоритм двигается перпендикулярно этим линиям в сторону уменьшения функции.

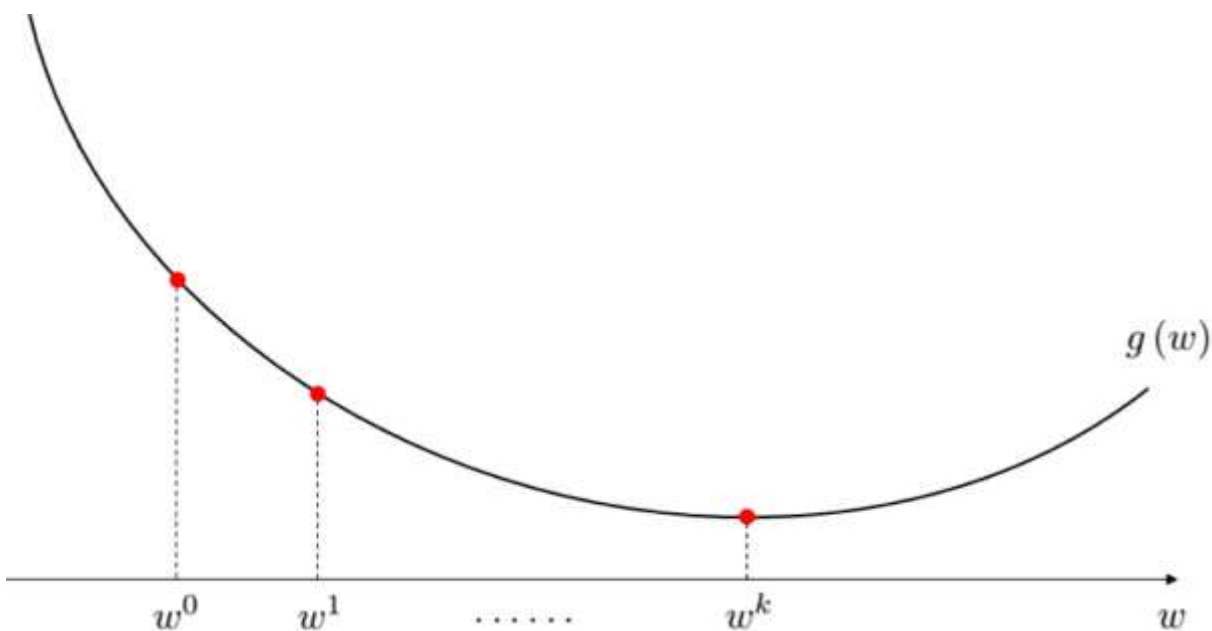


Рис. 4.1 — Геометрическая иллюстрация направления спуска.

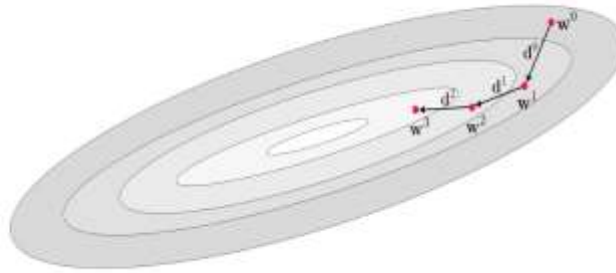


Рис. 4.2 — Траектория локальной оптимизации на линиях уровня.

4.7 Преимущества и ограничения

Преимущества локальных методов:

- высокая масштабируемость — не страдают от проклятия размерности;
- эффективность — сходимость часто достигается за малое число итераций;
- универсальность — применимы к широкому классу задач.

Ограничения:

- могут застревать в локальных минимумах;
- зависят от начальной точки;
- требуют корректного выбора длины шага α_k .

4.8 Вывод

Локальные методы оптимизации являются основой большинства современных алгоритмов машинного обучения. Они позволяют поэтапно приближаться к минимуму функции, используя правило итерационного обновления:

$$\mathbf{w}^{k+1} = \mathbf{w}^k + \alpha_k \mathbf{d}^k$$

Данная формула является универсальной — она лежит в основе как простых методов нулевого порядка (например, случайного и координатного поиска), так и более сложных градиентных и квазиньютоновских методов. В дальнейшем будут рассмотрены примеры конкретных реализаций, начиная со случайного поиска.

5. Случайный поиск

Случайный поиск (Random Search) является простейшим локальным методом оптимизации нулевого порядка. Он не требует вычисления производных и используется для нахождения минимума функции путём случайного выбора направлений движения. Метод демонстрирует основные принципы оптимизации: направление спуска, длину шага и последовательное улучшение решения.

5.1 Основная идея

На каждой итерации алгоритм выбирает несколько случайных направлений и проверяет, в каком из них значение функции уменьшается. Если найдено направление, ведущее к уменьшению, выполняется шаг в этом направлении. Таким образом, функция последовательно снижается до достижения локального минимума.

5.2 Алгоритм случайного поиска

Пошагово метод можно описать так:

1. Задать начальную точку w^0 и шаг $\alpha > 0$.
2. На k -й итерации сгенерировать P случайных направлений $d_1, d_2, \dots, d_P$.
3. Для каждого направления вычислить значение функции в новой точке:

$$f_i = f(w^k + \alpha d_i), \quad i = 1, 2, \dots, P$$

4. Найти направление с минимальным значением функции:

$$i^* = \arg \min f_i$$

5. Если найденное значение лучше текущего, перейти в новую точку:

$$w^{k+1} = w^k + \alpha d_{i^*}$$

6. Иначе выбрать новый набор направлений и повторить поиск.
7. Алгоритм останавливается, если значение функции перестаёт уменьшаться или достигнуто заданное число итераций.

5.3 Геометрическая интерпретация

На плоскости метод можно представить как выбор нескольких случайных стрел из текущей точки в разных направлениях. Из всех направлений выбирается то, где функция принимает меньшее значение. Далее точка смещается вдоль этого направления. Процесс повторяется, пока функция не перестанет уменьшаться.

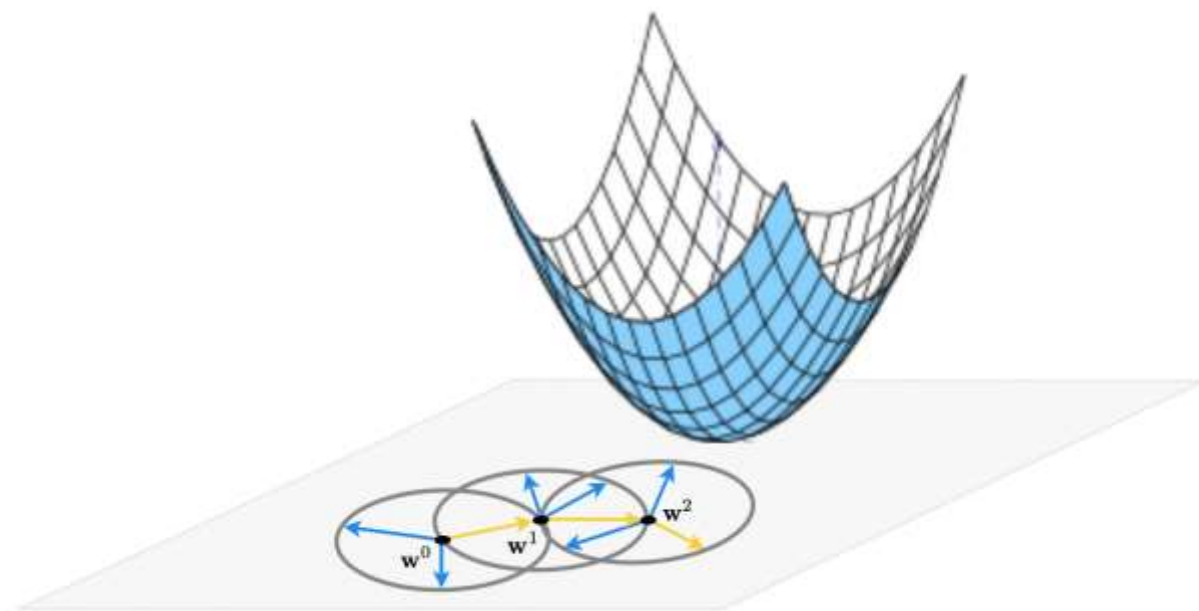


Рис. 5.1 — Иллюстрация случайного поиска на параболической поверхности.

5.4 Выбор направлений и нормализация

Чтобы направления были сопоставимы, обычно используют векторы единичной длины, то есть нормированные направления. Тогда длина шага определяется только параметром α :

$$\|w^{k+1} - w^k\| = \alpha$$

Направления часто выбираются случайно из нормального распределения с последующей нормализацией: $d_i \sim \mathcal{N}(0, I)$, затем $d_i \leftarrow d_i / \|d_i\|$.

5.5 Параметр шага и его настройка

Величина шага α играет ключевую роль. Если шаг слишком велик, алгоритм может перескочить через минимум; если слишком мал — сходимость будет крайне медленной.

Выделяют три типа настройки шага:

- фиксированный шаг: $\alpha_k = \text{const}$;

- убывающий шаг: $\alpha_k = 1 / (k + 1)$;
- адаптивный шаг — изменяется в зависимости от успешности итерации.

5.6 Пример: минимизация квадратичной функции

Рассмотрим функцию $f(x, y) = x^2 + y^2$. Пусть $w^0 = (2, 2)$, $\alpha = 0.5$, число направлений $P = 3$.

На каждой итерации генерируются три случайных направления d_1, d_2, d_3 . После нормализации проверяются точки $w^k + \alpha d_i$, вычисляются значения функции и выбирается минимальное. Если найдено улучшение, точка смещается в соответствующее направление. Последовательность точек постепенно приближается к $(0, 0)$, где находится глобальный минимум.

5.7 История функции стоимости

Для анализа сходимости удобно отслеживать последовательность значений функции по итерациям:

$$H(k) = f(w^k)$$

График $H(k)$ называется историей функции стоимости (cost function history). Он должен монотонно убывать, если алгоритм успешно приближается к минимуму. Такой подход удобен для визуализации работы локальных методов в многомерных пространствах.

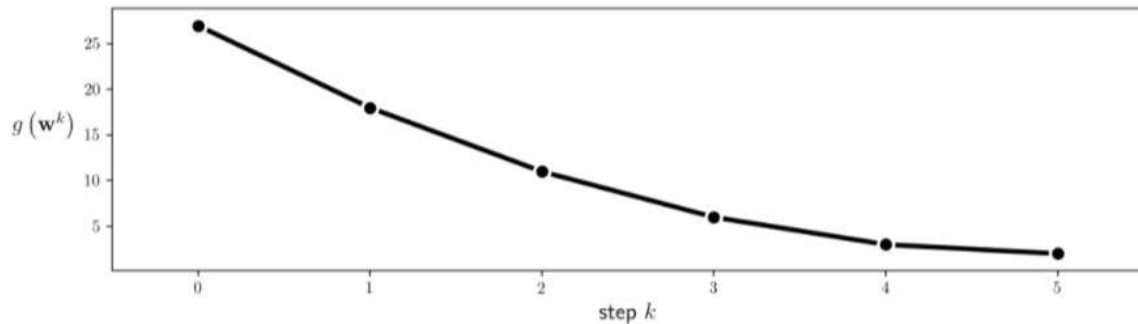


Рис. 5.2 — Пример истории функции стоимости.

5.8 Влияние размерности и вероятность спуска

Эффективность случайного поиска снижается с ростом размерности пространства параметров. Рассмотрим функцию $f(\mathbf{w}) = \|\mathbf{w}\|^2$, стартуя из точки $\mathbf{w}^0 = (1, 0, \dots, 0)$. В 1D вероятность выбрать направление спуска равна 50%. В 2D — около 43%, в 3D — менее 30%, а при $N = 10$ — меньше 1%. Таким образом, вероятность «угадать» правильное направление экспоненциально уменьшается с ростом N . Это одно из проявлений проклятия размерности.



Рис. 5.3 — Зависимость вероятности спуска от размерности пространства.

5.9 Преимущества и недостатки метода

Преимущества:

- простота реализации;
- устойчивость к шуму и разрывам функции;
- применимость к задачам типа «чёрного ящика».

Недостатки:

- высокая вычислительная стоимость при большом числе направлений;
- плохая масштабируемость по размерности;
- стохастический характер — разные запуски могут давать разные результаты.

5.10 Вывод

Случайный поиск является фундаментальным примером локальной оптимизации нулевого порядка. Он иллюстрирует идею поэтапного улучшения решения на основе сравнения значений функции в случайных направлениях. Хотя в практике машинного обучения метод применяется редко, он лежит в

основе более сложных алгоритмов — эволюционных стратегий, стохастических градиентных процедур и методов поиска по координатам. Его понимание важно для осознания общих принципов оптимизации без использования производных.

6. Координатный поиск и спуск

Координатный поиск и координатный спуск относятся к числу наиболее простых и эффективных методов нулевого порядка. В отличие от случайного поиска, где направления выбираются произвольно, здесь движение ограничивается только вдоль координатных осей. Такой подход уменьшает вычислительную сложность и делает алгоритм более предсказуемым.

6.1 Основная идея

Методы координатного типа строятся на идее последовательного улучшения значений функции вдоль отдельных координат. То есть вместо изменения всех параметров одновременно, мы оптимизируем функцию по одной переменной за раз, фиксируя остальные.

Если функция имеет вид $f(w_1, w_2, \dots, w_N)$, то на каждом шаге рассматривается изменение только одной координаты, например w_i , при фиксированных остальных значениях. Этот процесс можно рассматривать как серию одномерных оптимизаций.

6.2 Геометрическая интерпретация

В двумерном случае (x, y) движение вдоль координатных осей напоминает «лесенку»: сначала шаг вдоль оси x , затем вдоль оси y , и так далее. Каждый шаг уменьшает значение функции вдоль своей координаты, приближая точку к минимуму.

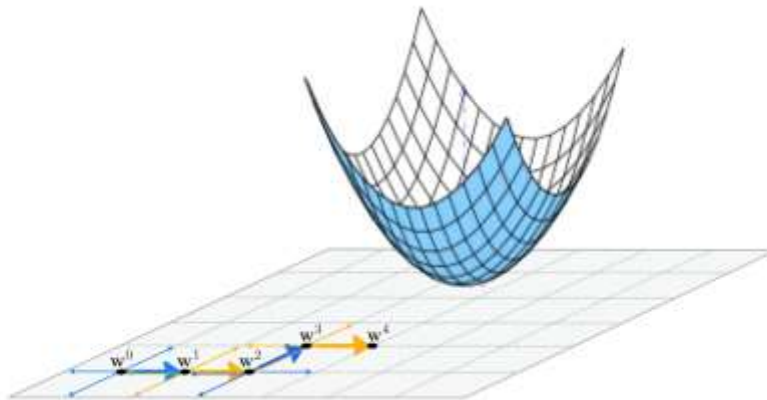


Рис. 6.1 — Иллюстрация траектории координатного спуска в двумерной функции.

6.3 Алгоритм координатного поиска

Координатный поиск (coordinate search) выполняет оценку функции вдоль всех координатных осей на каждом шаге. Набор направлений состоит из положительных и отрицательных единичных векторов стандартного базиса:

$$\mathfrak{D} = \{ \pm e_1, \pm e_2, \dots, \pm e_N \}$$

Для каждой координаты вычисляются два значения функции:

$$f(w^k + \alpha e_i), \quad f(w^k - \alpha e_i)$$

Затем выбирается направление, которое даёт наименьшее значение функции. После этого выполняется обновление точки:

$$w^{k+1} = w^k + \alpha d^k$$

где $d^k \in \mathfrak{D}$ — выбранное направление. Такой поиск требует $2N$ вычислений функции на каждой итерации.

6.4 Алгоритм координатного спуска

Координатный спуск (coordinate descent) — более экономичный вариант. Здесь координаты оптимизируются последовательно: вместо того, чтобы проверять все направления сразу, на каждом шаге изменяется только одна координата.

Пусть $w^k = (w_1^k, w_2^k, \dots, w_N^k)$. Для координаты i выполняется проверка:

$$f(w^k + \alpha e_i) < f(w^k) \rightarrow w_i^{k+1} = w_i^k + \alpha$$

Если уменьшение не достигнуто, координата остаётся без изменения. После обхода всех координат получаем новое приближение w^{k+1} .

Координатный спуск требует такого же числа вычислений функции, как координатный поиск ($2N$ на итерацию), но часто обеспечивает более быстрое уменьшение функции, так как обновления выполняются чаще.

6.5 Пример: функция двух переменных

Рассмотрим функцию $f(w_1, w_2) = w_1^2 + 4w_2^2$. Начальная точка $w^0 = (2, 2)$, шаг $\alpha = 0.5$.

Шаг 1: изменяем w_1 , проверяя $f(1.5, 2)$ и $f(2.5, 2)$. Минимум достигается при $w_1 = 1.5$.

Шаг 2: изменяем w_2 , проверяя $f(1.5, 1.5)$ и $f(1.5, 2.5)$. Минимум достигается при $w_2 = 1.5$.

После двух шагов получаем точку $(1.5, 1.5)$, которая ближе к глобальному минимуму $(0, 0)$. Такой процесс продолжается, постепенно приближая решение к оптимуму.

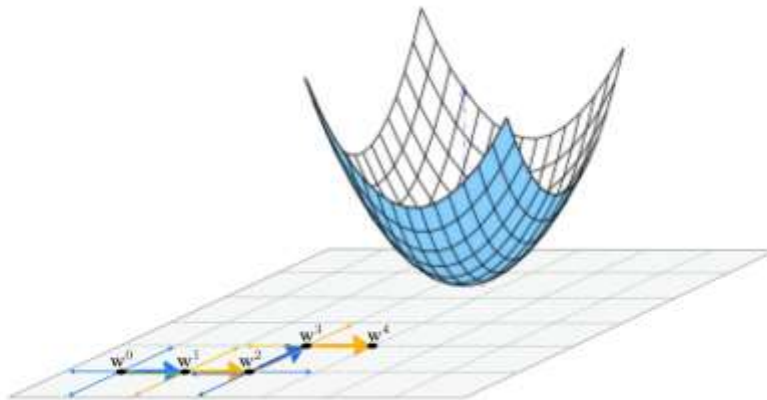


Рис. 6.2 — Траектория движения координатного спуска.

6.6 Сравнение методов

Координатный поиск и спуск превосходят случайный поиск по эффективности, так как рассматривают ограниченный набор направлений. Однако из-за ограниченности направлений (только осевые векторы) может потребоваться больше итераций для достижения точного минимума.

Сравнение подходов:

- Случайный поиск — произвольные направления, высокая вариативность, низкая эффективность;
- Координатный поиск — систематические направления, предсказуемое поведение;
- Координатный спуск — последовательное обновление координат, быстрая сходимость при малых вычислительных затратах.

6.7 Практические особенности

Преимущества координатных методов:

- простая реализация и малые требования к памяти;
- масштабируемость при большом числе переменных;
- возможность параллельного обновления нескольких координат.

Недостатки:

- возможна медленная сходимость, если переменные сильно коррелированы;
- ограниченность направлений поиска (только вдоль осей).

Тем не менее координатные методы широко применяются в машинном обучении, особенно при обучении моделей с разреженными параметрами (например, в линейной и логистической регрессии).

6.8 Вывод

Методы координатного поиска и спуска являются эффективной альтернативой случайным методам нулевого порядка. Они позволяют избежать избыточных вычислений за счёт ограничения направления поиска и обеспечивают хорошую сходимость при умеренной размерности задач. Эти методы послужили основой для целого класса современных алгоритмов координатной оптимизации, включая градиентные и стохастические варианты.

7. Сравнение методов и заключение

Методы оптимизации различаются по порядку информации, используемой о функции: методы нулевого порядка опираются только на значения

функции, методы первого — на её производные (градиент), а методы второго — дополнительно учитывают вторые производные (гессиан).

7.1 Классификация методов оптимизации

Общая классификация выглядит следующим образом:

Порядок метода	Используемая информация	Типичные примеры	Преимущества	Недостатки
Нулевой	Значения функции $f(w)$	Случайный поиск, координатный поиск	Простота, устойчивость к шуму, не требует производных	Медленная сходимость, плохая масштабируемость
Первый	Градиент $\nabla f(w)$	Градиентный спуск, стохастический градиент	Быстрая сходимость, эффективен в ML-задачах	Требуется вычисления или оценки градиента
Второй	Гессиан $\nabla^2 f(w)$	Метод Ньютона, квазиньютоновские методы	Квадратичная сходимость, высокая точность	Большая вычислительная сложность, хранение матриц

Из таблицы видно, что методы нулевого порядка являются наиболее универсальными, но менее эффективными. Методы первого и второго порядков дают более быструю сходимость, но требуют дополнительной информации о функции.

7.2 Роль методов в машинном обучении

В задачах машинного обучения оптимизация используется для настройки параметров моделей, минимизируя функцию потерь. Наиболее часто применяются методы первого порядка, например стохастический градиентный спуск (SGD):

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \alpha_k \nabla f(\mathbf{w}^k)$$

Однако в ситуациях, когда функция потерь недифференцируема, содержит шум или вычисление градиента невозможно (например, при работе с симуляциями, моделями «чёрного ящика» или разрывными функциями), используются методы нулевого порядка — в частности, случайный и координатный поиск, а также эволюционные алгоритмы.

7.3 Взаимосвязь между порядками

Методы нулевого порядка формируют интуитивную основу для понимания всех остальных подходов. Переход к первому порядку добавляет информацию о направлении наискорейшего спуска, а второй порядок — о кривизне функции, что ускоряет сходимость.

Таким образом, можно рассматривать иерархию методов оптимизации как постепенное наращивание знаний о функции: от простого сравнения значений — к учёту наклонов и, наконец, формы поверхности.

7.4 Выводы по лекции

1. Методы нулевого порядка — это фундаментальные инструменты оптимизации, применимые даже без аналитических выражений функции.
2. Они обеспечивают простоту и надёжность, но требуют большого числа вычислений.
3. Методы первого и второго порядков развивают эти идеи, используя производные и вторые производные для ускорения сходимости.

4. В машинном обучении все эти подходы объединяются в гибридные схемы, где сочетаются принципы нулевого, первого и второго порядков.

7.5 Заключение

Методы нулевого порядка оптимальности — это отправная точка в изучении теории и практики оптимизации. Они формируют основу понимания понятий минимума, направления спуска и длины шага. Хотя их эффективность ограничена для задач высокой размерности, они остаются незаменимыми для анализа и обучения интуитивным идеям. В дальнейшем развитие методов оптимизации приводит к созданию мощных адаптивных алгоритмов, лежащих в основе современного машинного обучения.